

Claude Sonnet 5 vs Opus 4.8

Three One-Shot Build Prompts

A simple prompt pack for testing how two Claude models perform across design, app functionality, and complex game logic.

1	Portfolio Website	Tests design, copy, layout, responsiveness, and polish.
2	Personal Finance App	Tests forms, calculations, local storage, validation, and dashboard UX.
3	Castle Defense Game	Tests game logic, waves, movement, scoring, upgrades, and win/loss states.

Use the same prompt for both models, in the same environment, with the same rules. Then compare the final results side by side.

How to Use This Prompt Pack

Run each test as a one-shot build. Give Claude Sonnet 5 and Claude Opus 4.8 the exact same prompt, then compare the final output without giving one model extra help.

Recommended setup: React + Tailwind CSS, same editor or coding environment, same prompt, same amount of follow-up help, and the same manual testing checklist.

Test	What It Measures	Score
Portfolio Website	Design quality, copy, responsiveness, spacing, visual polish	/10
Personal Finance App	Functionality, calculations, local storage, validation, usability	/10
Castle Defense Game	Game loop, enemy movement, defenders, scoring, upgrades, win/loss logic	/10

A simple final question for the video: Is Opus 4.8 still clearly better, or is Sonnet 5 now good enough for most people?

Prompt 1: Portfolio Website

Goal: test design taste, copywriting, layout, responsiveness, and polish.

Build a modern personal portfolio website for a freelance designer and developer named Alex Morgan.

Use React and Tailwind CSS.

The website should feel premium, clean, and modern. It should look like something a real creative professional could use to get clients.

Create a full landing page with the following sections:

1. Hero section
 - Strong headline
 - Short subheadline
 - Two call-to-action buttons
 - A visual area that shows a profile card, project preview, or creative mockup
2. About section
 - Short professional bio
 - Skills and tools
 - A clean layout that feels personal but professional
3. Featured projects section
 - Show 4 projects
 - Each project should have a title, category, short description, and visual card
 - Add hover effects
4. Services section
 - Show 3 services:
 - Website Design
 - Web Development
 - Brand Identity
5. Testimonials section
 - Include 3 realistic client testimonials
6. Contact section
 - Contact form with name, email, project type, and message
 - Add basic form validation
 - Show a success message after submission
7. Footer
 - Include social links and copyright text

Design requirements:

- Premium portfolio style
- Clean spacing
- Modern typography
- Fully responsive for desktop, tablet, and mobile
- Smooth hover effects
- Subtle animations
- Good visual hierarchy
- Make it look polished, not like a basic template

Technical requirements:

- Use React components
- Use Tailwind CSS

-
- Keep everything frontend-only
 - No backend required
 - No external APIs
 - Make sure the app does not crash
 - Include comments for important sections

At the end, include a manual testing checklist for:

- Desktop responsiveness
- Mobile responsiveness
- Contact form validation
- Success message after form submission
- Hover effects
- All sections rendering correctly

Prompt 2: Personal Finance Web App

Goal: test forms, calculations, state management, local storage, validation, and useful dashboard design.

Build a functional personal finance web app called BudgetFlow.

Use React and Tailwind CSS.

The app should help users track income, expenses, savings goals, and monthly budget progress.

Core features:

1. Dashboard overview

- Show total income
- Show total expenses
- Show remaining balance
- Show savings progress
- Show spending by category

2. Add transaction form

The user should be able to add a transaction with:

- Transaction name
- Amount
- Type: income or expense
- Category
- Date
- Notes

3. Transaction list

- Show all transactions
- Allow users to filter by income, expense, or category
- Allow users to search transactions by name
- Allow users to delete a transaction

4. Budget categories

Create default categories:

- Housing
- Food
- Transportation
- Entertainment
- Shopping
- Health
- Education
- Other

5. Savings goal tracker

- User can set a savings goal name
- User can set a target amount
- User can add money toward the goal
- Show percentage progress
- Show how much is left to reach the goal

6. Charts and visual summaries

- Add a simple chart or visual breakdown for spending by category
- Add a monthly income vs expense summary
- Use simple frontend visuals if a chart library is not available

7. Local storage

- Save transactions to local storage
- Save savings goal to local storage

- Data should still be there after refresh

8. Validation

- Do not allow empty transaction names
- Do not allow invalid amounts
- Do not allow negative amounts
- Show helpful error messages

9. UI requirements

- Clean dashboard style
- Modern cards
- Clear labels
- Responsive layout
- Easy to understand
- Works well on desktop and mobile

Technical requirements:

- Use React state properly
- Use clean component structure
- Use Tailwind CSS
- Keep everything frontend-only
- No backend required
- No external APIs
- Make sure the app does not crash during normal use
- Add comments for the main logic

At the end, include a manual testing checklist for:

- Adding income
- Adding expenses
- Deleting transactions
- Filtering transactions
- Searching transactions
- Budget totals updating correctly
- Savings goal updating correctly
- Local storage persistence after refresh
- Form validation
- Mobile responsiveness

Prompt 3: Castle Defense Game

Goal: test complex one-shot building with game logic, waves, movement, scoring, upgrades, and win/loss states.

Build a complete browser-based game called Castle Defense.

Use React and Tailwind CSS.

Game concept:

The player is defending a castle from waves of enemies. Enemies move from the left side of the screen toward the castle on the right side. The player places defenders on the battlefield to stop the enemies before they reach the castle.

The game should feel like a simple tower defense game.

Core gameplay:

- Enemies move from left to right across the game board
- If an enemy reaches the castle, the player loses one life
- The player starts with 20 lives
- The player starts with 150 gold
- The player earns gold when enemies are defeated
- The game has 10 waves
- Each wave should become harder
- The player wins after surviving all 10 waves
- The player loses if lives reach 0

Enemy types:

1. Goblin
 - Fast movement
 - Low health
 - Reward: 10 gold
2. Orc
 - Medium speed
 - Medium health
 - Reward: 20 gold
3. Troll
 - Slow movement
 - High health
 - Reward: 35 gold
4. Dark Knight
 - Appears in later waves
 - High health
 - Medium speed
 - Reward: 50 gold

Defender types:

1. Archer
 - Low cost
 - Fast attack speed
 - Low damage
2. Knight
 - Medium cost
 - Medium attack speed
 - Medium damage
3. Mage

- Higher cost
- Slower attack speed
- Area damage

4. Guardian

- Expensive
- Boosts nearby defenders

Required features:

- Start screen with game explanation
- Game board
- Defender selection panel
- Place defenders by selecting one and clicking on the board
- Do not allow placement if the player does not have enough gold
- Enemies should move across the board
- Defenders should automatically attack enemies within range
- Show attack animations or visual effects
- Wave system
- Gold system
- Lives system
- Score system
- Defender upgrade system
- Win screen
- Game over screen
- Restart button
- Pause button
- Simple sound toggle button, even if using visual-only sound state
- Activity log that explains what is happening during the game

UI requirements:

- Make the game look polished and fun
- Use a fantasy castle theme
- Clear game board
- Clean cards and buttons
- Show current wave, lives, gold, and score
- Show defender stats before placing them
- Show enemy health visually
- Responsive layout for desktop and mobile

Technical requirements:

- Keep everything in one working React app
- Use React state properly
- Use clean component structure
- Avoid external APIs
- Avoid backend requirements
- Use local state only
- Add comments for the main game logic
- Make sure the game does not crash during normal play

At the end, include a manual testing checklist for:

- Starting the game
- Placing each defender type
- Preventing placement without enough gold
- Enemies moving correctly
- Defenders attacking enemies
- Enemy health decreasing
- Gold increasing after enemies are defeated
- Lives decreasing when enemies reach the castle
- Waves getting harder
- Upgrading defenders

-
- Pausing and restarting the game
 - Winning after wave 10
 - Losing when lives reach 0
 - Mobile responsiveness